Worksheet: Journey to wastewateR Session

Michigan Public Health Integrated Center for Outbreak Analytics and Modeling (MICOM)

Getting Started

1. Go to https://github.com/micom-hub/insightnet_2026_wastewateR_tools_session and download the **`compiled_data`** folder, and the **`data_files`** folder.
2. Open RStudio and open a blank code file.
3. Install / load in the following libraries:
    a. wastewateR - https://micom-hub.org/wastewateR_documentation/
    b. tidyverse - https://tidyverse.org/

Introduction to Laboratory Data Format - One File

Welcome to the laboratory! Congratulations on running your first **five ddPCR** plates testing samples from **6 wastewater treatment plants**! Your five data files look great. Our local health department messaged us yesterday, and they think that the **respiratory disease season** should be winding down. Let's take a look at our testing results and see if we can give them any actionable information!

1. Using R, read in the **`03.25.26_GT_fiveplex1_QX600.csv`** data file to a dataframe called **`one_file`**.
2. Use **`colnames(one_file)`** to view the list of column headers in the file
3. Use **`table(one_file$Sample.description.1)`** to view all the sample names present in this file

🦆What do you notice about these sample names? Can you identify which names might be laboratory controls and which ones might be samples from sites?

4. Use **`table(one_file$Target)`** to view all the pathogen targets that were tested for in this plate run.

🦆What targets are being tested for in this plate of data?

These test results are from a GTMolecular kit that can test for multiple pathogens in the same well. For simplicity, the BRSV results have been removed from these data files. For more information on this, please visit:
https://www.gtmolecular.com/product/gt-digital-flusc2rsv-wastewater-surveillance-panel-v1-0-for-the-bio-rad-qx600-droplet-digital-pcr-system/

5. Let's create a plot to show the plate layout
    a. First, split the **`Well`** column into rows (letter) and numbers (columns)
    b. Start your **`ggplot`** with **`x = columns`**, and **`y = rows`**, with a **`geom_tile`**, filled by **`Positives`**. Add a **`geom_text`** that labels each tile with the number of Positive Droplets (**`Positives`**).

🦆What do you notice about the plot so far? (Number of wells, labelling clarity?)

    c. Let's **`facet_wrap()`** our plot by Target, and use **`scale_y_discrete(limits=rev)`** to re-order our y-axis.

🦆What wells do you think the positive controls are in?

Processing Laboratory Data for QAQC - One File

Now that we've familiarized ourselves with the data, let's use the [wastewateR package](https://micom-hub.org/wastewateR_documentation/main_functions_processing.html#stand-alone) to do quality assurance / quality control (QAQC) on our lab's data. We'll process each file individually.

1. Use **`list.files()`** to get a vector of all the **`.csv`** laboratory data files you have.
2. Read in the first file in your vector

```
file_in <- read.csv(paste0(data_file_loc, lab_files[1]))
```

3. Next, we need to use the **`software_renaming()`** [https://micom-hub.org/wastewateR_documentation/main_functions_processing.html#stand-alone] function to slim down our dataset (we don't need every column to perform QAQC) and to standardize our naming convention of each column. Save the new dataframe in a separate variable, so you can compare the two.

🦆What is different in the new dataframe? Can you identify which new columns were mapped from the original dataframe?

4. We're going to use the **`qaqc_processing_fiveplex()`** function [https://micom-hub.org/wastewateR_documentation/main_functions_processing.html#wrappers] from the wastewateR package to perform QAQC.
    a. We'll have to provide some extra information to the wrapper function in the **`lab_site_ids`** parameter, in order for it to process properly
        i. Our lab_site_ids are = "26BB", "26CC", "26HH", "26JJ", "23MM", "26LL", "23ZZ". These are all the acceptable sample name codes that identify the site that the sample came from. This is always the first four characters of our sample names.

🦆Review the messaging output from the function. What parts of the messaging output do you understand? Which do you not?

Note: This message output can be ported to a text file, when this code is used in an automated pipeline system, to generate processing logs.

b. The structural output from this function is a list of two dataframes. Save each of the dataframes in its own variable, so you can view them.

Calculations and Data Display

To make things a little easier, I've compiled all of the processed data into one file for us to use. That file is available as **`post_process_compiled.csv`** . Now we need to create charts to understand what the measured pathogen levels have been in our samples from each testing site.

1. Read in our **`post_process_compiled.csv`** data as a data frame.
2. In order to calculate the Gene Copies per 100mL in a sample, we need some measurements from the lab that are taken before the laboratory ddPCR testing. Read in the **`pre_processing_data.csv`** file as a dataframe.
3. We need a CP/uL column to use in our gene copies per 100mL calculation. For this data, the CP/uL calculation from the laboratory methodology is:

```
CP_uL = CopiesPer20uLWell / 5
```

4. Use **`calculate_gc_per_100ml()`** to calculate our gene copies per 100mL and the associated limit of detection values
    a. We did not use any dilutions, so we can leave that input parameter of the function alone!

🦆Pay attention to the message that appears. Was there any data that has been lost in this calculation?

5. Use **`table()`** to determine how many rows per Sample and Target combination there are in this data.
    a. Then, use **`group_by()`** and **`summarize()`** to calculate the average Sample value and Limit of Detection value for each Sample and Target combination.

```
gc_calc_one_value <- gc_calculations %>%
group_by(plate_name, Sample, Target) %>%
summarize(mean_limit_of_detection = mean(detection_limit_CP_100mL, na.rm = TRUE),
          mean_gc_per_100mL = mean(CP_100_mL_of_sample, na.rm = TRUE))
```

6. Use **`mutate()`** to create an indicator variable for samples that are above or below the limit of detection.

```
gc_calc_one_value <- gc_calc_one_value %>%
mutate(relative_lod = case_when(mean_gc_per_100mL > mean_limit_of_detection ~ "above",
                                                                  T ~ "below"))
```

7. Get the sample collection date and collection site out of the sample name, assigning it to a new column in your dataframe.

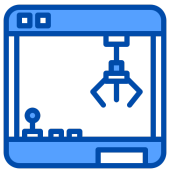
```
gc_calc_one_value <- gc_calc_one_value %>%
mutate(sample_collection_date = as.POSIXct(substr(Sample, 5, 10), format = "%y%m%d"),
       collection_site = substr(Sample, 1, 4))
```

8. Make a graph of your results, using **`ggplot()`** with **`geom_point`**, assigning the shape aesthetic to your limit of detection indicator, coloring the plot points by collection site, and faceting the plot by Target.

🦆Has there been any respiratory pathogen activity detected via wastewater in the past month? What other data might be helpful to build context around this information?

Note: This is not the only way to display wastewater-based epidemiological data! Another way to display data is to show concentration values normalized by PMMoV. You've possibly seen either of these metrics used, depending on your source of wastewater-based epidemiological data.

Bonus Section! Displaying Data as Concentration per uL



This section offers a look at a different way to both display data (in different units) and to calculate a limit of detection. We'll use the same dataset that we started with in the previous section (**`post_process_compiled.csv`**) and go from there.

1. Read in our **`post_process_compiled.csv`** data as a data frame.
2. Select all negative control samples (NEG) from that data frame, and save them in a separate dataframe. Select the columns: Sample, Target, plate_name, Positives, AcceptedDroplets.
3. Using the plate_name column, create a new column called plate_date that contains the date the plate run was from.
4. Select the columns in your negative control data frame in this order: Sample, Target, plate_name, plate_date, Positives, AcceptedDroplets; and rename the columns to: "Sample", "Target", "plate", "plate_date", "Positives", "AcceptedDroplets"
5. Use **`calculate_lod_invgamm()`** to create a limit of detection dataframe from your negative control dataframe.
6. Use **`merging_methods()`** to turn the full dataset (from #1) from having multiple lines per Sample and Target pair, to having a single line per Sample and Target pair.
    a. Note: you lose the plate information when you do this! In order to maintain that information to be able to join the dataframe to the limit of detection dataframe, create a bridge table. Select only distinct rows of Sample, Target, and plate_name.
7. Join the dataframes (your limit of detection dataframe, your bridge table, and your merged sample dataset)

8. Use **`mutate()`** to create an indicator variable for samples that are above or below the limit of detection.

```
final_conc_set <- final_conc_set %>%
mutate(relative_lod = case_when(Concentration > NEG_limit_final ~ "above",
                                                                T ~ "below"))
```

9. Get the sample collection date and collection site out of the sample name, assigning it to a new column in your dataframe.

```
final_conc_set <- final_conc_set %>%
mutate(sample_collection_date = as.POSIXct(substr(Sample, 5, 10), format = "%y%m%d"),
       collection_site = substr(Sample, 1, 4))
```

10. Make a graph of your results, using **`ggplot()`** with **`geom_point`**, assigning the shape aesthetic to your limit of detection indicator, coloring the plot points by collection site, and faceting the plot by Target.

🦆How does this chart compare to the gene copies per 100mL display? Try changing the y-axis limits with **`coord_cartesian()`**.

🦆 Bonus: Compare the two sets of data - does one limit of detection technique result in different above/below calls than the other? How might that impact the interpretation of wastewater results?

A document of completed code (**`sample_answers.html`**) can be found at:

https://github.com/micom-hub/insightnet_2026_wastewateR_tools_session/tree/main/answer_key

The answer key code does not display the only way to code these solutions, just one of many.

Citations:

- https://www.flaticon.com/free-icons/scientist Scientist icons created by Freepik - Flaticon “scientist icons"
- https://www.flaticon.com/free-icons/student Student icons created by Darius Dan - Flaticon "student icons”
- https://www.flaticon.com/free-icons/claw-machine Claw-machine icons created by xnimrodx - Flaticon "claw-machine icons"